Contours & Shape Analysis

Week 9 — From regions to measurable objects

Practical Image Processing

Lecture 9 ▪ 60–70 min

Recap & today

- Wk8: segmentation gives separated regions/blobs.
- Today: **contours** — the outlines of those regions — and how to **measure** shape: area, perimeter, bounding box, and shape descriptors.

DocScan milestone

Find each digit's contour, crop it by its bounding box, and order the digits — the last step before reading (Week 13).

Roadmap

Finding contours

Measuring shape

Shape descriptors

In-class exercise

Deep dive

Wrap-up

Finding contours

A contour = the outline of a region



binary digits



contours (red)

- A **contour** is an ordered list of boundary points of a connected white region.
- From a clean binary image, `findContours` traces each object's outline.
- Each contour is one object you can measure and crop.

findContours: modes & approximation

```python
contours, hierarchy = cv2.findContours(binary,
    cv2.RETR_EXTERNAL,        # only outer contours (ignore holes)
    cv2.CHAIN_APPROX_SIMPLE)  # compress straight segments to endpoints
```

- **RETR_EXTERNAL**: outer boundaries only. **RETR_TREE**: full nesting hierarchy.
- **CHAIN_APPROX_SIMPLE**: stores only corner points (a rectangle = 4 points, not hundreds).

Contours vs connected components

- **Connected components** label each pixel with its region id (a labelled image).
- **Contours** give the ordered *boundary* of each region (a list of points).
- Use components to count/label; contours to measure shape and crop. Often used together.

Key idea

Both turn a binary mask into a list of objects — the bridge from pixels to countable, measurable things.

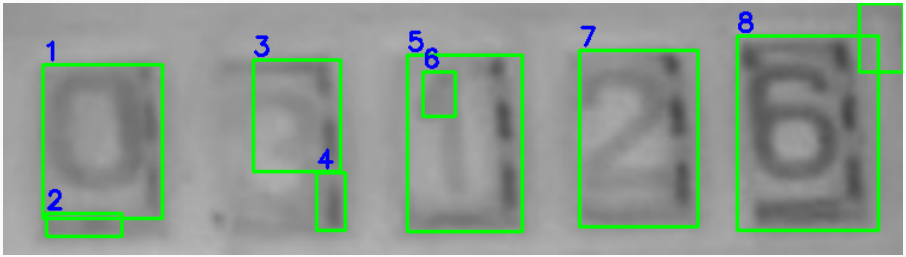
Measuring shape

Basic measurements

Measurement	OpenCV
Area	`cv2.contourArea(c)`
Perimeter	`cv2.arcLength(c, True)`
Bounding box (axis-aligned)	`cv2.boundingRect(c)`
Rotated box	`cv2.minAreaRect(c)`
Enclosing circle	`cv2.minEnclosingCircle(c)`
Centroid	from `cv2.moments(c)`

These turn a shape into numbers you can filter, sort, and compare.

Bounding boxes → crop & order the digits



- Draw a box around each digit; sort boxes by their x-coordinate (left to right).
- Crop each box → one clean image per digit, *in order*.
- This is exactly the DocScan hand-off to the recognizer (Week 13).

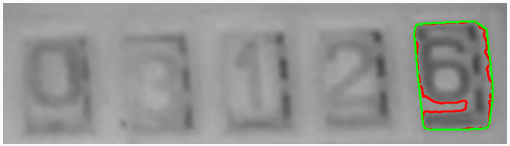
Filtering by shape

- Real images give junk contours (noise specks, partial marks). Filter them out by:
- **Area** (too small = noise), **aspect ratio** (digits are taller than wide), **extent** (fill fraction).
- Keep only contours that look like digits — a few thresholds remove most false positives.

✔ Good practice

"Keep contours with area $>$ T and $0.2 < \mathrm{w/h} < 1.0$" filters noise from digits robustly.

Convex hull & convexity



contour (red) + convex hull (green)

- **Convex hull**: the smallest convex shape enclosing the contour (a rubber band).
- **Convexity defects** (dents) reveal fingers, gaps, or touching-object necks.
- **Solidity** = area / hull area — how "filled" a shape is.

Shape descriptors

Descriptors: numbers that describe a shape

- **Circularity** $= 4\pi\,\text{area}/\text{perimeter}^2$ (1 for a circle, lower for complex shapes).
- **Aspect ratio**, **extent** (area/box area), **solidity** (area/hull area).
- **Moments**: weighted sums that summarize a shape (centroid, orientation, spread).

Math

Centroid: $\bar{x} = M_{10}/M_{00}$, $\bar{y} = M_{01}/M_{00}$, where M_{00} is the area.

Hu moments: rotation/scale-invariant shape

- **Hu moments**: 7 numbers that are (nearly) invariant to **translation, rotation, and scale**.
- So the same shape gives the same descriptor regardless of where/how big/rotated it is.
- Useful for matching a shape to a template — a classical alternative to a neural net.

✓ Good practice

`cv2.matchShapes` compares two contours via Hu moments — a quick shape similarity score.

Contour approximation (polygon fitting)

- `approxPolyDP` simplifies a contour to fewer points within a tolerance.
- A page/document boundary → approximates to *4 points* = the corners.
- Those 4 corners feed the perspective correction in Week 11 — the DocScan "flatten" step.

Key idea

Approximating the page contour to a quadrilateral is how DocScan finds the page's corners.

Contour hierarchy (holes)

- **RETR_TREE** returns a hierarchy: which contour is inside which (a digit "0" has an outer and an inner contour).
- Useful to count holes (a "0" and "8" have them; a "1" doesn't) — a cheap feature.
- For simple digit cropping, **RETR_EXTERNAL** (outer only) is usually enough.

In-class exercise

In-class exercise (12 min)

Design the filter

You have contours from a meter photo: 9 digits plus some noise specks and a long scratch.

1. Which measurements distinguish a digit from a speck? From a scratch?
2. Write a filter rule (area + aspect ratio) that keeps digits and drops the rest.
3. After filtering, how do you output the number in the correct order?

Debrief

- Speck: tiny area → drop by `area > T`. Scratch: extreme aspect ratio (very wide/thin) → drop.
- Rule: keep if `area>200 and 0.2<w/h<1.2` — digits are compact and tallish.
- Order: sort the kept bounding boxes by x; read left to right.

Key idea

Shape measurements turn "which blobs are digits?" into a few numeric thresholds.

Deep dive

Counting & measuring objects

- Count = number of valid contours (after filtering) — cells, coins, parts, digits.
- Measure = area/length/diameter per contour, converted to real units via a known scale.
- This is the backbone of industrial inspection and microscopy — pure classical vision.

✔ Good practice

A calibrated pixel-to-mm ratio turns contour measurements into real-world dimensions.

Robustness: the real-world contour mess

- Contours are only as good as the binary mask — garbage in, garbage out.
- Broken digit $\rightarrow$ two contours; merged digits $\rightarrow$ one. Fix upstream (Weeks 7–8).
- Always filter by shape *and* sanity-check the count on real data.

✕ Pitfall

Don't trust the contour count blindly — a shadow or a smudge can add or merge objects.

Contours in code — the full loop

```python
cnts, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
digits = []
for c in cnts:
    x, y, w, h = cv2.boundingRect(c)
    if cv2.contourArea(c) > 200 and 0.2 < w/h < 1.2:   # keep digit-like
        digits.append((x, gray[y:y+h, x:x+w]))
digits = [img for x, img in sorted(digits)]            # left-to-right
```

Segment → contour → filter → crop → order. The DocScan hand-off, in seven lines.

Fourier descriptors (brief)

- Represent a contour as a signal and take its Fourier transform (Week 6 again).
- Low-frequency coefficients = overall shape; high = fine boundary detail.
- Keeping a few coefficients gives a compact, robust shape descriptor for matching.

Shape analysis → recognition

- Hu moments / descriptors can classify simple shapes without any training.
- For digits, hand-crafted descriptors work but a CNN (Week 13) is more accurate.
- The pipeline is the same: segment → contour/crop → describe/classify.

Key idea

Contours give you clean, isolated objects; classical descriptors or a CNN then name them.

Wrap-up

What you should now be able to do

- Find contours; choose retrieval mode and approximation.
- Measure area/perimeter/bounding box/centroid; compute shape descriptors.
- Filter contours by shape; use Hu moments for invariant matching.
- Approximate a page contour to 4 corners; crop & order digits.

This week's deliverable

Due before Week 10 (see `project/`)

1. Find and draw contours on a cleaned meter binary; report each digit's area & box.
2. Filter out noise contours with an area + aspect-ratio rule; verify the count.
3. Crop each digit by its bounding box and output them ordered left-to-right.
4. Bonus: approximate a document's outer contour to its 4 corners.

Contours turn regions into measurable objects.

Find, measure, filter, crop, order — the last step before reading.